

WCW Memory Monitor

WinterCrownWORKS · Unreal Engine 向けリアルタイムメモリ / VRAM 監視プラグイン · v0.2.0 (2026-09-28)

1. 概要

WCW Memory Monitor は、プロジェクトに入れる **.uplugin** 形式の軽量 Slate オーバーレイです。単体アプリではありません。物理 RAM、PC VRAM、テクスチャストリーミングプール、TextureGroup ごとの Resident、選択した LLM タグ、任意の RHI リソース枠に加え、クック済みビルドのキャプチャをチェンジリストと突き合わせるためのビルドスタンプを表示します。

2. 導入のメリット

- プレイ中に OOM リスクやテクスチャ予算超過を、製品プレイヤーへ届く前に拾う。
- プロジェクト構成に合わせた TextureGroup / LLM だけを監視する。
- オーバーレイはいつでも ON/OFF。重い RHI 走査はオプトインなので、既定では安く保つ。
- TitleVersion + CL + UE パッチをキャプチャに焼き、QA ショットが自己説明的になる。

3. 表示サンプル



CitySample。黄文字が Config/Wcw.ini 由来のビルドスタンプ。左が Quality Presets。RHI 行は Wcw.MemoryMonitor.EnableRHI 0 のあと Use: --- MB (Disabled)。

4. 主な機能

HUD	半透明 Slate。HitTestInvisible のため操作を奪わない。
System / Pool	物理 RAM、Windows では DXGI Dedicated VRAM、ストリーミングプールの Required vs Pool。
TextureGroup	グループごとの Resident。予算超過は赤、ピークホールド付き。
LLM	選択した ELLMTag。起動引数 -11m が必要。
RHI	RHIGetTrackedResourceStats の走査 (Lumen / Nanite / Shadow / VT など)。フレームが重いときは EnableRHI で切る。

ビルドスタンプ	Wcw.ini の TitleVersion + CL。無ければ ProjectVersion。最後に 1.0.0.0。UE パッチは常に付く。
---------	--

5. 導入 : Plugins へ配置

.uproject と同じ階層の Plugins へリポジトリ一式をコピーします。フォルダ名は変えないでください。

```
YourProject/
  YourProject.uproject
  Config/
    DefaultEngine.ini
    DefaultGame.ini
    Wcw.ini          ← プロジェクト側のスタンプ（プラグイン同梱ではない）
  Plugins/
    WcwMemoryMonitorModule/
      WcwMemoryMonitorModule.uplugin
    Source/   Resources/
```

1. プロジェクトを開き直す。C++ ならプロジェクトファイル再生成とコンパイル。
2. [編集] → [プラグイン] で「WCW Memory Monitor」または「WcwMemoryMonitorModule」を有効化。必要なら再起動。
3. 全プロジェクト共有なら Engine/Plugins/ でも可。通常はプロジェクトの Plugins/ が安全。

6. クイックスタートとコンソールコマンド

PIE / Standalone / Development パッケージのゲーム内コンソール (～) から実行します。

Wcw.MemoryMonitor.ToggleUI	オーバーレイの表示切替。初回 ON でウィジェット生成。
Wcw.MemoryMonitor.FontScale 1.2	プロジェクト設定の FontSize に掛ける倍率。既定 1.0。Tick で即反映。
Wcw.MemoryMonitor.EnableRHI 0 1	RHI リソース追跡の無効 / 有効。1 で追跡済み RHI を走査し Lumen / Nanite / Shadow / VT などの行を埋める。0 では走査せず、それらの行は「Use: --- MB (Disabled)」。

FontScale やプロジェクト設定の多くは DefaultEngine.ini に残ります。EnableRHI は実行時 CVar なので、見たいセッションに合わせて切り替えてください。

LLM と実行環境

LLM: 正確な LLM 行には起動引数 -llm が必要です。無いと 0 のままです。

PIE: エディタ内プレイのヒープは実機と一致しません。予算サインオフは Standalone か実機パッケージで行ってください。

7. ビルドスタンプ — Wcw.ini・フォールバック・クック前書き込み

黄文字は SWcwMemoryBudgetWidget.cpp の MakeBuildLabel() です。ウィジェット生成時 (ToggleUI ON) に一度だけ解決します。

バージョン文字列の優先順

1. <Project>/Config/Wcw.ini の [Wcw.Build] / **TitleVersion**
2. 同じファイルの **ProjectVersion** (TitleVersion が空のときの別名)
3. DefaultGame.ini の [/Script/EngineSettings.GeneralProjectSettings] **ProjectVersion**
4. 最後の手段 1.0.0.0

Wcw.ini に空でない **Changelist** があれば CL ... を足します。エンジン側は常に FEngineVersion::Current() のパッチまでです。

表示例

```
Build: 0.0.1 | CL 12345 | UE 5.8.0    ← Wcw.ini あり (今回の CitySample)
Build: 0.9.3 | CL 88421 | UE 5.5.4    ← CI が TitleVersion + Changelist を書いた場合
Build: 1.0.0.0 | UE 5.5.4            ← Wcw.ini 無し、ProjectVersion も空
```

Wcw.ini テンプレート（ゲームプロジェクトへ置く。プラグイン配下ではない）

```
; WinterCrownWORKS shared project stamp.
; CI / PreBuild がクック前に書く。プラグインは読むだけ。
; パス: <Project>/Config/Wcw.ini
```

```
[Wcw.Build]
TitleVersion=0.9.3
Changelist=88421
Stamp=2026-09-28T16:00:00+09:00
Dirty=0
```

```
[Wcw.Common]
Studio=WinterCrownWORKS
Project=
```

プラグインリポジトリに置いただけでは実行時に見えません。読み先は `FPaths::ProjectConfigDir() / "Wcw.ini"` です。ファイルが無ければ失敗して Project Settings に落ちます。

推奨する PreBuild / CI の書き込み

クック機で CL を手編集しないでください。UAT / エディタクックの直前にパイプラインが Wcw.ini を出すのが安全です。Perforce ならチェンジリスト番号、Git なら `git rev-list --count HEAD` と短い SHA、あるいはタイトルの版付けスクリプト。ローカル用に `.uproject / Target.cs` の PreBuild から書いても構いません。

```
# 例: RunUAT BuildCookRun の前 (GitHub Actions / Jenkins)
CL=$(git rev-list --count HEAD)
VER=${TITLE_VERSION:-0.9.3}
STAMP=$(date -Iseconds)
cat > "$PROJECT_DIR/Config/Wcw.ini" <<EOF
[Wcw.Build]
TitleVersion=$VER
Changelist=$CL
Stamp=$STAMP
Dirty=0
EOF
# その後クック。Config/Wcw.ini がパッケージに残るか確認する。
```

パッケージ時の注意

Default*.ini 以外の独自 ini は、クックの Config フィルタで落ちることがあります。実機に Config/Wcw.ini が残っているか確認してください。無ければ HUD は黙って ProjectVersion / 1.0.0.0 に落ちます。ウィジェットの不具合ではなく、フォールバック動作です。

8. RHI 追跡を切ることがある理由

EnableRHI は `RHIGetTrackedResourceStats` をゲートします。追跡中の全 RHI リソースを歩き、名前で Lumen / Nanite / Shadow / DistanceFields / IndexBuffer / VertexBuffer / VirtualTexture / Hair に振り分けます。CitySample のような密なシーンではフレーム時間が目に見えて伸びます。EnableRHI 0 で約 45 ms だった同じ画角が、1 にした直後の走査フレームで 100–200 ms 超になることがあります。

- 数値が欲しいとき: `Wcw.MemoryMonitor.EnableRHI 1` — Lumen / Nanite / Shadow / VT 行に MB が入る。
- オーバーレイを安くしたいとき: `Wcw.MemoryMonitor.EnableRHI 0` — それらの行は Use: --- MB (Disabled)。テクスチャ / LLM のピークホールドは動いたまま。
- `RHI_ENABLE_RESOURCE_INFO` 付きビルドが前提。Test / Development 向け。Shipping では走査対象が空になりがちです。
- HUD に出すのは non-transient サイズのみです。

9. ピークホールドと QA 動画

TextureGroup と LLM の行は、使用量が予算を超えた瞬間の値を約 5 秒残し、(Peak: 1753.1 MB @ 21:41:06) のように **MB と壁時計** を出します。スパイクは 1 フレームで消えることが多いので、生の Use だけだとスクショも動画も「今は収まっている」に見えます。ホールドがあるとそのピークが画に残ります。

後から QA が動画だけ送ってきたとき、突き合わせの軸は次の 2 本です。

- 画面左上のビルドスタンプ (TitleVersion / CL / UE) — どの提出物か。
- 赤行の Peak 時刻 — そのセッション内のいつか。ログの LogTemp や LLM / memreport のタイムスタンプと同じ時計です (FDateTime::Now())。

動画を止めて Peak の時分秒を読み、同じ時刻付近のログを開けば、「どの CL の、どの秒で、どのグループが予算を超えたか」がキャプチャー一枚で足ります。RHI を切っていても、このホールドは Texture / LLM 側では動きます。

10. プロジェクト設定

[編集] → [プロジェクト設定] → [Plugins] → [WCW Memory Monitor]。内容は DefaultEngine.ini に保存されます。

- FontSize — 基準ポイント。FontScale CVar が倍率。
- SystemGroups / TargetGroups / LLMGroups / RhiResourceGroups — 出す行。エディタ上で重複は自動で空き枠に振り直されます。
- Texture / LLM 行の BudgetMB が赤表示と 5 秒ピークホールドの閾値です。

設定デモ: <https://www.youtube.com/watch?v=o5P6D3sfGsk>

11. ライセンス

MIT License。使用・改変・再配布は自由です。Repository: github.com/yuukajii/UnrealWcwMemoryMonitor

© 2026 WinterCrownWORKS.